Lab Practical 2 (Windows)

To Develop a MapReduce program to calculate the frequency of a given word in a given file.

Run this ONLY ONCE when setting up Hadoop for the first time!

Command:

hdfs namenode –format

Starting Hadoop Services (Every Time)

Always open CMD as Administrator before running these commands!

Step 1: Start HDFS

Command:

start-dfs.cmd

Step 2: Start YARN

Command:

start-yarn.cmd

Step 3: Exit Safe Mode

Command:

hdfs dfsadmin -safemode leave

Step 4: Check Running Services

jps

Practical 2

Step 1: Create Working Directory
Commands:

```
mkdir C:\wordcount
cd C:\wordcount
```

Step 2: Create Input File
Command:

```
notepad input.txt
```

Paste this content into the file:

```
bus,train,Bus,TRAIN,car,Bus,Train,TRAIN,Car,car,Bus,Train,CAR,bus,BUs,TrAin,bus
```

Step 3: Create WordCount.java
Command:

```
notepad WordCount.java
```

Paste this code:

```
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
```

```
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {
  public static void main(String[] args) throws Exception {
    Configuration c = new Configuration();
    Job j = Job.getInstance(c, "wordcount");
    j.setJarByClass(WordCount.class);
    j.setMapperClass(WordCount.MapForWordCount.class);
    j.setReducerClass(WordCount.ReduceForWordCount.class);
    j.setOutputKeyClass(Text.class);
    j.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(j, new Path(args[0]));
    FileOutputFormat.setOutputPath(j, new Path(args[1]));
    System.exit(j.waitForCompletion(true) ? 0 : 1);
  }

  public static class MapForWordCount extends
      Mapper<LongWritable, Text, Text, IntWritable> {
    public void map(LongWritable key, Text value, Context con)
        throws IOException, InterruptedException {
      String line = value.toString();
      String[] words = line.split(",");
      for (String word : words) {
        Text outputKey = new Text(word.toUpperCase().trim());
        IntWritable outputValue = new IntWritable(1);
        con.write(outputKey, outputValue);
      }
    }
  }

  public static class ReduceForWordCount extends
      Reducer<Text, IntWritable, Text, IntWritable> {
    public void reduce(Text word, Iterable<IntWritable> values,
        Context con) throws IOException, InterruptedException {
      int sum = 0;
      for (IntWritable value : values) { sum += value.get(); }
      con.write(word, new IntWritable(sum));
    }
  }
}
```

Step 4: Compile the Java Program
Command:

```
javac -classpath "%HADOOP_HOME%\share\hadoop\common\hadoop-common-
3.3.6.jar;%HADOOP_HOME%\share\hadoop\mapreduce\hadoop-mapreduce-client-core-
3.3.6.jar;%HADOOP_HOME%\share\hadoop\common\lib\commons-cli-1.2.jar" WordCount.java
```

Step 5: Create JAR File
Command:

```
jar cf wordcount.jar WordCount*.class
```

Step 6: Upload Input File to HDFS

Commands:

```
hadoop fs -put C:\wordcount\input.txt /
hadoop fs -ls /
```

Step 7: Delete Old Output (if exists)
Command:

```
hadoop fs -rm -r /wc_output
```

Step 8: Run the MapReduce Job
Command:

```
hadoop jar wordcount.jar WordCount /input.txt /wc_output
```

Step 9: View the Output
Command:

```
hadoop fs -cat /wc_output/part-r-00000
```

Stopping Hadoop Services

Always stop Hadoop properly before closing CMD windows!

Step 1: Stop YARN

stop-yarn.cmd

Step 2: Stop HDFS

stop-dfs.cmd